第 5 讲　关联规则

关联规则定义

支持度和置信度

频繁项集

关联规则的误区

Apriori算法

现实案例分析



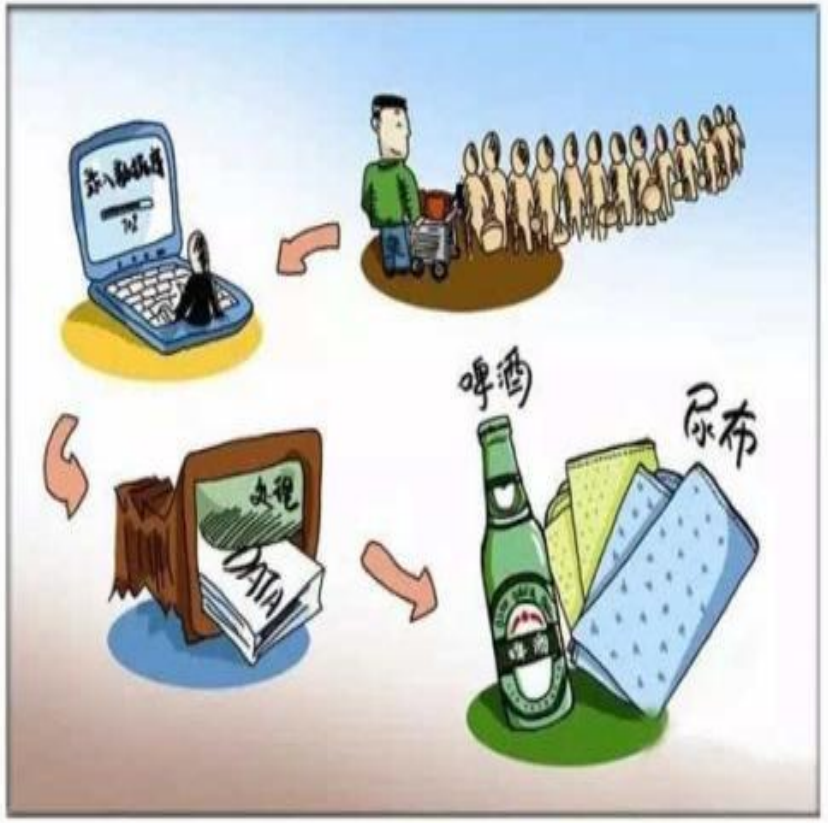
什么是关联规则挖掘?

- **关联规则挖掘：**
  - **发现事务数据库, 关系数据, 或其它信息库中项或数据对象集合间的频繁模式, 关联, 相关, 或因果关系结构.**
  - **频繁模式: 在数据库中频繁出现的模式(项集, 序列, 等)**
- **动机：发现数据中的规律性（购物篮分析）**
  - **哪些产品更经常一起购买? — 啤酒 和 尿布?!**
  - **购买了PC后, 哪些将相继购买?**
  - **什么类型的 DNA 对新药敏感?**
  - **我们能自动地对Web文档分类吗?**



“尿布与啤酒”——隐藏的典型关联分析案例

早在20世纪80年代，沃尔玛超市就已经将关联规则应用到了商品管理之中。沃尔玛曾经对数据仓库中一年多的原始交易数据进行了详细的分析，发现许多美国家庭都是妻子在家照顾婴儿，丈夫去超市为婴儿买尿布。丈夫们在购买尿布时往往会顺便买两瓶啤酒来犒劳自己。这一现象引起了沃尔玛的重视，沃尔玛调整了货架的位置，把尿布和啤酒摆在相邻的位置，以便于年轻的爸爸们能顺利地同时找到这两种商品。这一故事中的“啤酒”与“尿布”的关系即为所谓的“关联性”，而“关联性”的发掘和利用则是本章所要讨论的“关联规则挖掘”。



关联规则概述

- 频繁项集（Frequent Itemsets）
- 关联规则（Association Rules）













真实案例：

Frequently Bought Together



Price For All Three: $166.83

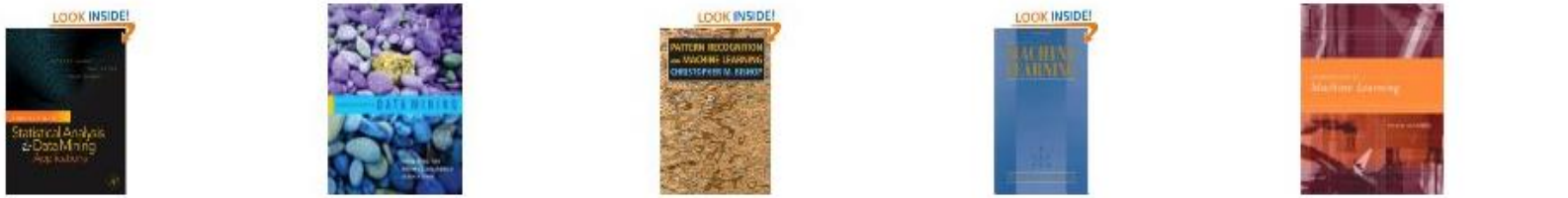
Add all three to Cart | Add all three to Wish List

Show availability and shipping details

频繁项集问题

- **This item:** Data Mining: Practical Machine Learning Tools and Techniques, Second Edition (Morgan Kaufmann Series in Data Management Systems) by Eibe Frank
- The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition (Springer Series in Statistics) by Robert Tibshirani
- Pattern Recognition and Machine Learning (Information Science and Statistics) by Christopher M. Bishop

Customers Who Bought This Item Also Bought



Handbook of Statistical Analysis and Data Mining... by John Elder IV
(9) $71.96

Introduction to Data Mining by Pang-Ning Tan
(15) $80.80

Pattern Recognition and Machine Learning... by Christopher M. Bishop
(47) $58.03

Machine Learning (Mcgraw-Hill International Edit) by Tom M. Mitchell
(38) $73.72

Introduction to Machine Learning (Adaptive Comp... by Ethem Alpaydin
(8) $43.79

关联规则问题



基于市场的分析

- **在事务数据库中查找项目之间的关联**
- **项目**（Items）
  - Bread, Milk, Chocolate, Butter …
- **交易、事务**（Transaction (Basket) ）
  - 所有项目的非空子集
- 交叉销售（Cross Selling），向上销售（Up Selling）
  - 向现有客户销售其他产品或服务
- 捆绑折扣（Bundle Discount）
- 商品布局设计（Shop Layout Design）
  - Minimum Distance vs. Maximum Distance
- **“Baskets” & “Items”: Sentences & Words**





需掌握的基本概念

- 引出之后需要的几个概念：





关联规则定义

- 某个事务是一组项目：$T \subseteq \{ I = \{ i_a, i_b, \ldots, i_t \} \}$
- T 是 I 的子集，其中 I 是所有可能项的集合
- 数据集 D 包含一组事务 T

$$D \subseteq \{ T = \{ T_a, T_b, \ldots, T_t \} \} \subseteq \{ I = \{ i_a, i_b, \ldots, i_t \} \}$$

- **关联规则的形式是：**

$$P \Rightarrow Q \quad \text{where } P \subset I, Q \subset I \text{ and } P \cap Q = \varnothing$$

- **一组项目被称为项目集（*Itemset*）**
- **一个项目集包含k个项目的称为（*K-itemset*）**
- **项目集可以看作是项目的结合**



事务（Transactions）

Transactions	Items
1	Bread, Jelly, Peanut, Butter
2	Bread, Butter
3	Bread, Jelly
4	Bread, Milk, Butter
5	Chips, Milk
6	Bread, Chips
7	Bread, Milk
8	Chips, Jelly



Searching for rules in the form of: Bread → Butter



支持度（Support of an Itemset）

- 项目(或项目集) X 的支持是该项目(或项目集)发生的事务的百分比

$$Support(X) = \frac{\#X}{n}$$

- 支持度s
  - 规则 $X \Rightarrow Y$ 在数据集D中的支持度为s, 其中s 表示D中包含X∪Y (即同时包含X和Y)的事务的百分率，即概率 $P(X \cup Y)$

则称关联规则X⇒Y的支持度为s%

$$Support(X \Rightarrow Y) = P(X \cup Y)$$



支持度（Support of an Itemset）

- **项目(或项目集) X 的支持是该项目(或项目集)发生的事务的百分比**

$$Support(X) = \frac{\#X}{n}$$

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

支持度单调性：项目集的支持度永远不会超过它的子集的支持度，这被称为支持度单调性。 (Support of an itemset never exceeds the support of its subsets, this is known as the anti-monotone property of support)



支持度（Support of an Itemset）

- **项目(或项目集) X 的支持是该项目(或项目集)发生的事务的百分比**

$$Support(X) = \frac{\#X}{n}$$

Itemset	Support	Itemset	Support
Bread	6/8	Bread, Butter	3/8
Butter	3/8	…	
Chips	2/8	Bread, Butter, Chips	0/8
Jelly	3/8	…	
Milk	3/8	Bread, Butter, Chips, Jelly	0/8
Peanut	1/8	…	
		Bread, Butter, Chips, Jelly, Milk	0/8
		…	
		Bread, Butter, Chips, Jelly, Milk, Peanut	0/8



支持度和置信度

- 关联规则是支持 X→Y 是 D 中同时包含 X 和 Y 的事务数与总事务数的百分比

$$Support(X \rightarrow Y) = \frac{\#(X \cup Y)}{n}$$

- 关联规则的置信 *(Confidence)* X→Y 是 D 中同时包含 X 和 Y 的事务数与只包含 X 的事务数的比值

$$Confidence(X \rightarrow Y) = \frac{\#(X \cup Y)}{\#(X)}$$

- 综上公式：置信度可表示为

$$Confidence(X \rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)}$$

- 规则 X→Y 在数据集 D 中的可信度为 c，其中 c 表示 D 中包含 X 的事务中也含有Y的百分率，即可用条件概率 *P(Y|X)* 表示：

条件概率： *P(Y|X)* = $Confidence(X \rightarrow Y)$



支持度和置信度

- **支持度**：衡量 在数据集中规则出现的**频率**
- **置信度**：衡量 规则的**强度**

Transactions	Items
1	Bread, Jelly, Peanut, Butter
2	Bread, Butter
3	Bread, Jelly
4	Bread, Milk, Butter
5	Chips, Milk
6	Bread, Chips
7	Bread, Milk
8	Chips, Jelly

Bread → Milk

Support: 2/8

Confidence: 1/3

Milk → Bread

Support: 2/8

Confidence: 2/3



相关概念解读

- **Lift(提升度)：** ***P(X,Y)/P(X)P(Y)***，*Lift=1* 时表示*X*和*Y*独立。这个数越大(>1)，越表明*X*和*Y*存在于一个购物篮中不是偶然现象，有较强的关联度.

  Lift(X→Y) = P(Y|X) / P(Y) = Confidence(X→Y) / P(Y)

- **Leverage(杠杆率)：** ***P(X,Y)-P(X)P(Y)***，*Leverage=0* 时*X*和*Y*独立，*Leverage*越大*X*和*Y*的关系越密切

- **Conviction(确信度)：** ***P(X)P(!Y)/P(X,!Y)***，(*!Y*表示*Y*没有发生) *Conviction*也是用来衡量*X*和*Y*的独立性。从它和*lift*的关系(对*Y*取反，代入*Lift*公式后求倒数)可以看出，这个值越大，*A*、*B*越关联。



频繁项集和强规则

- **支持度和置信度一般给它们事先设定好阈值：**
  - 最小支持度 σ
  - 最小置信度 Φ
- **一个频繁项集的支持度必须大于支持度阈值 σ**
- **一个强规则首先它要是频繁的且规则置信度要高于阈值 Φ**
- **关联规则问题**
  - **得到项集 I、数据集 D、确定最小支持度 σ 和 最小置信度 Φ，去挖掘所有的强关联规则X→Y**
- **但是现实生活中往往可能的关联规则的数目是巨大的**
  - **蛮力策略是不可行的.**
  - **首先必须得使用一些聪明的方法去找到频繁项集（Frequent Itemsets）**



关联规则-整体框架

- **Step 1: 找出所有频繁项集**
- **Step 2: 使用频繁项集产生关联规则**
  - 对于每个频繁项集 f
    - 创建频繁项集 f 的所有非空子集
  - 对于 f 的每个非空子集 s
    - 输出 s → (f-s)　　当 support (f) / support (s) > Φ

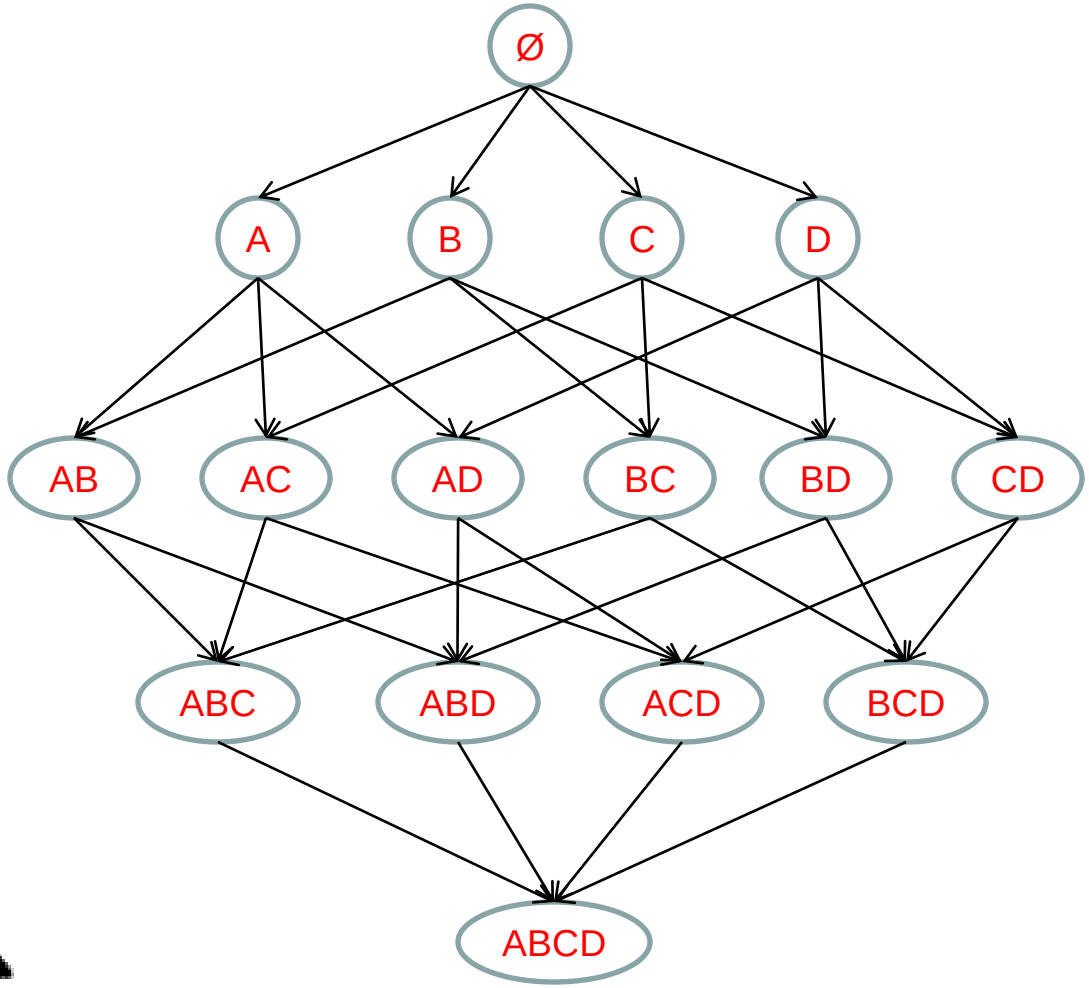
{a, b, c}

- a b → c
- a c → b
- b c → a
- a → b c
- b → a c
- c → a b

关键是找到频繁项集

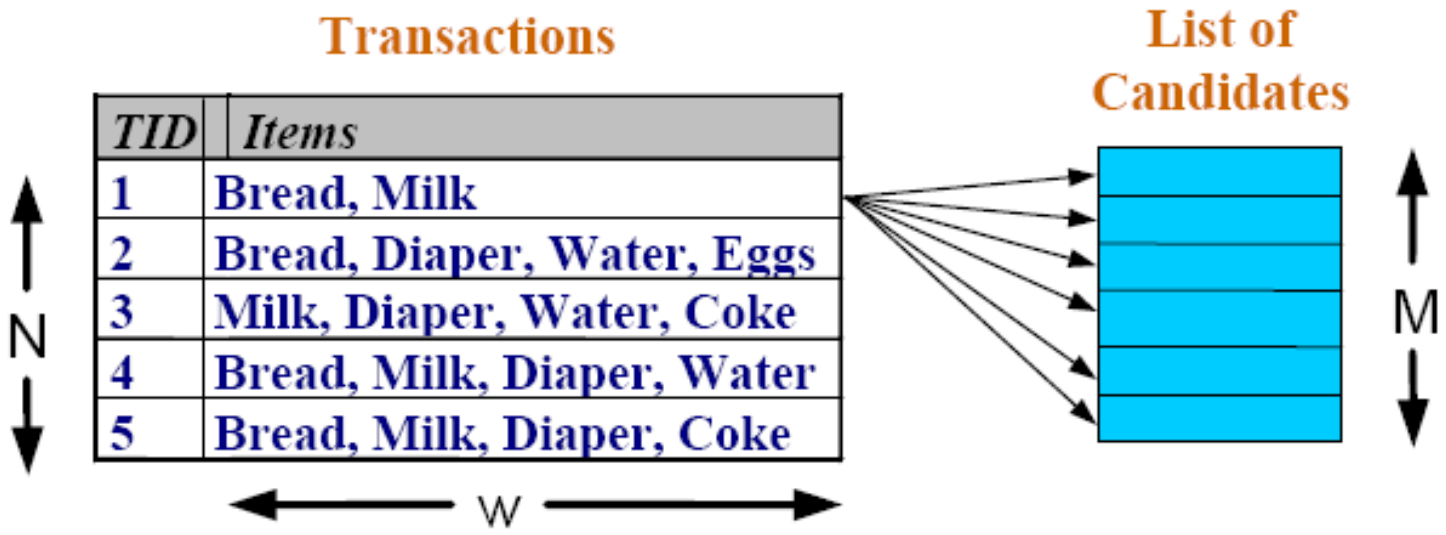


项集产生（Itemset Generation）





算法复杂度（Algorithm Complexity）





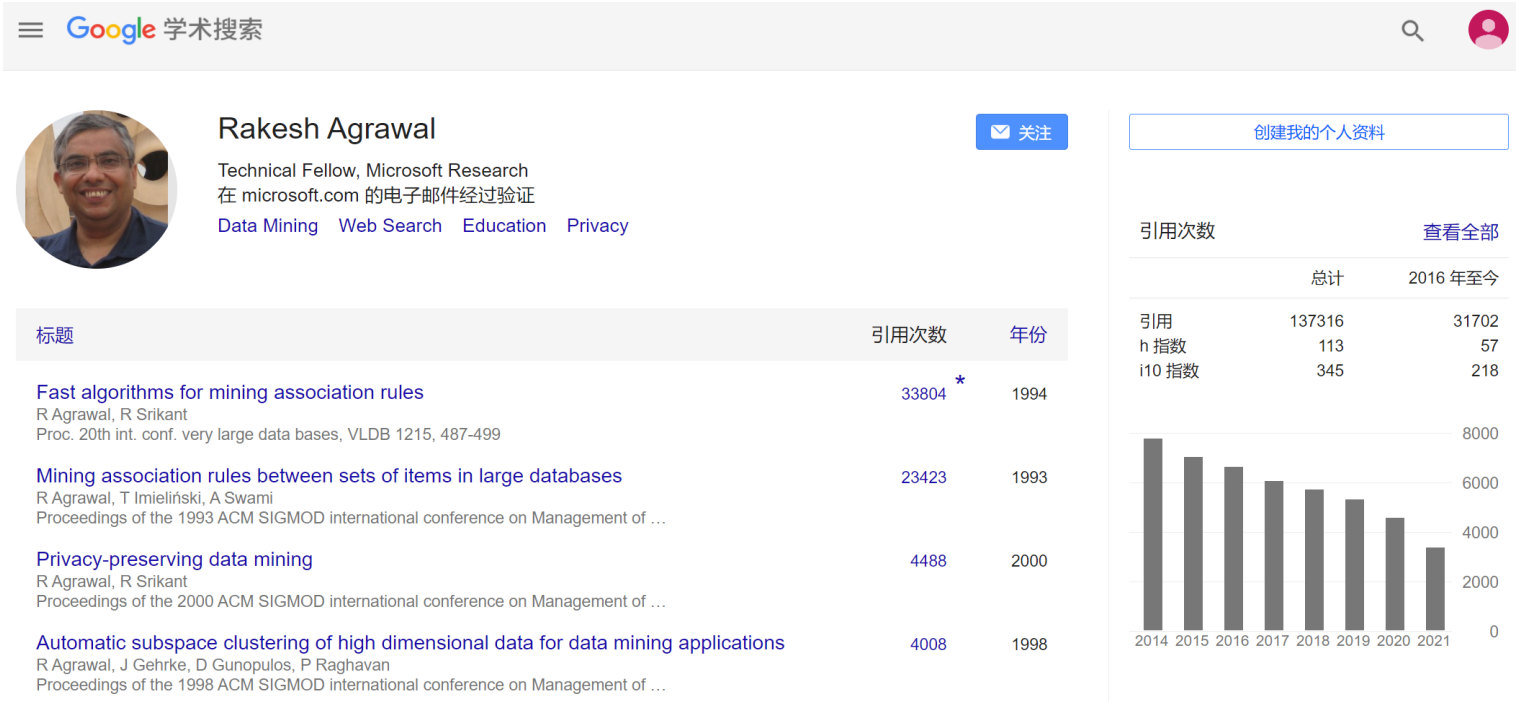
$$O(NMW)$$

$$M = 2^d - 1$$



Apriori 算法

- **数据挖掘中最著名的算法之一**
- **关键思想：**
  - **频繁项集的子集必须频繁**
    - {Milk, Bread, Coke} 频繁 → {Milk, Coke} 频繁
  - **任何不频繁项集的超集都不能频繁**
    - {Battery} 不频繁 → {Milk, Battery} 不频繁



Google 学术搜索

Rakesh Agrawal

关注

Technical Fellow, Microsoft Research
在 microsoft.com 的电子邮件经过验证
Data Mining Web Search Education Privacy

创建我的个人资料

标题	引用次数	年份
Fast algorithms for mining association rules R Agrawal, R Srikant Proc. 20th int. conf. very large data bases, VLDB 1215, 487-499	33804 *	1994
Mining association rules between sets of items in large databases R Agrawal, T Imieliński, A Swami Proceedings of the 1993 ACM SIGMOD international conference on Management of …	23423	1993
Privacy-preserving data mining R Agrawal, R Srikant Proceedings of the 2000 ACM SIGMOD international conference on Management of …	4488	2000
Automatic subspace clustering of high dimensional data for data mining applications R Agrawal, J Gehrke, D Gunopulos, P Raghavan Proceedings of the 1998 ACM SIGMOD international conference on Management of …	4008	1998

引用次数 查看全部

	总计	2016 年至今
引用	137316	31702
h 指数	113	57
i10 指数	345	218



Apriori 算法



Trevor Hastie — 被引用次数：285558
Professor of Statistics, Stanford University
在 stanford.edu 的电子邮件经过验证
Statistical learning and modeling　data mining　machine learning



Jiawei Han — 被引用次数：214558
Abel Bliss Professor of Computer Science, University of Illinois
在 cs.uiuc.edu 的电子邮件经过验证
data mining　database systems　data warehousing　information networks



Philip S. Yu — 被引用次数：146860
Professor of Computer Science, University of Illinons at Chicago
在 cs.uic.edu 的电子邮件经过验证
Data mining　Database　Privacy



Rakesh Agrawal — 被引用次数：137316
Technical Fellow, Microsoft Research
在 microsoft.com 的电子邮件经过验证
Data Mining　Web Search　Education　Privacy



Jeffrey Ullman — 被引用次数：133712
Professor of Computer Science, Stanford University
在 stanford.edu 的电子邮件经过验证
Database Systems　Data Mining



Sergey Senkin — 被引用次数：132783
IARC, WHO
在 who.int 的电子邮件经过验证
Computational analysis　statistics　data mining　cancer genomics　particle physics



Vipin Kumar — 被引用次数：111719
University of Minnesota
在 umn.edu 的电子邮件经过验证
Data mining　parallel computing　high performance computing　Artificial Intelligence
machine learning



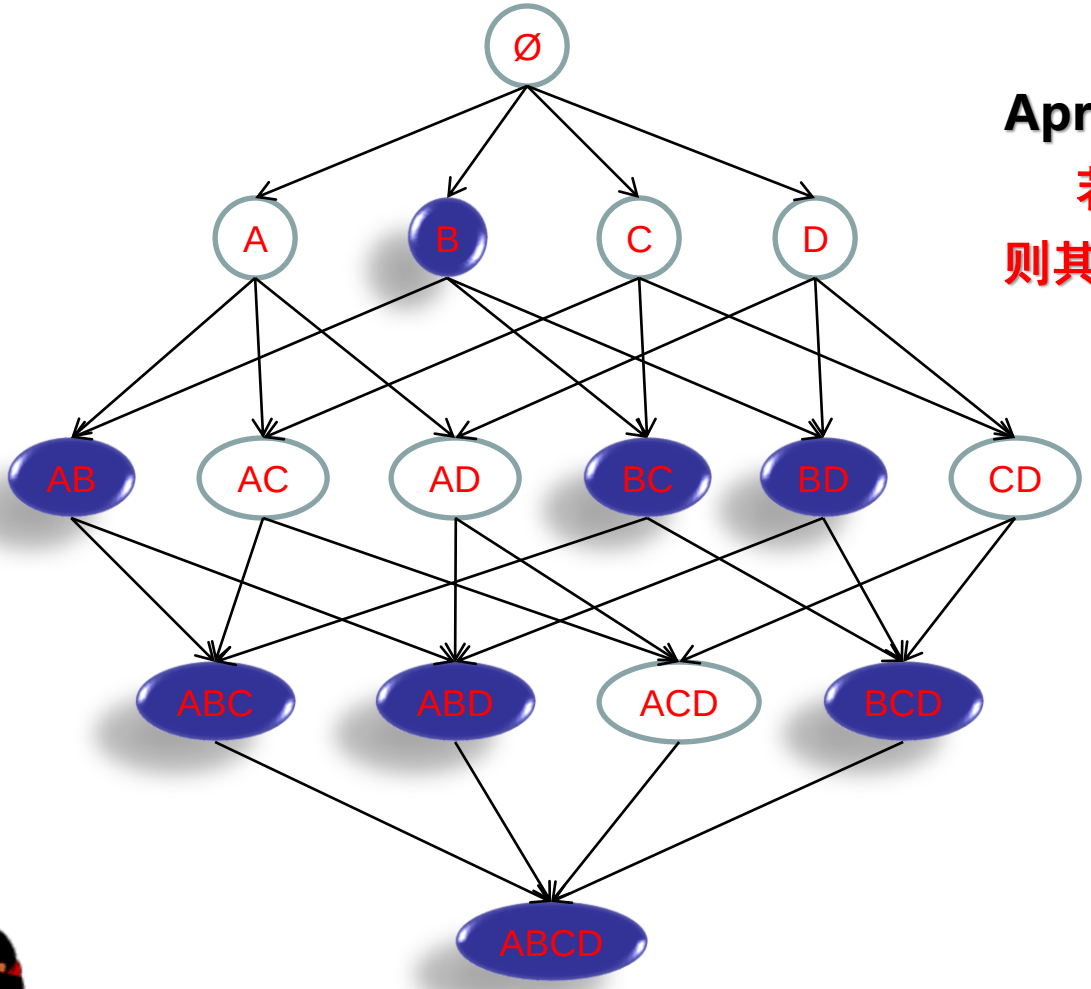
Jian Pei — 被引用次数：108670
Professor of Computing Science, Simon Fraser University
在 cs.sfu.ca 的电子邮件经过验证
Data mining　big data analytics　database systems　information retrieval



Chih-Jen Lin — 被引用次数：106317
Professor of Computer Science, National Taiwan University
在 csie.ntu.edu.tw 的电子邮件经过验证
Machine learning　Data Mining　Optimization　Artificial Intelligence



候选项剪枝



Apriori 剪枝原理:

若任一项集是不频繁的,则其超集不应该被生成







频繁项集的性质：

- **性质1：频繁项集的子集必为频繁项集**
- **性质2：非频繁项集的超集一定是非频繁的**
- Apriori算法运用性质：通过已知的频繁项集构成长度更大的项集，并将其称为潜在（候选）频繁项集。潜在频繁k-项集的集合C_k是指由有可能成为频繁k项集的项集组成的集合。
- 以后只需计算潜在频繁项集的支持度，而不必计算所有不同项集的支持度，因此在一定程度上减少了计算量。



Apriori算法

- 首先，通过扫描数据库，累积每个项的计数，并收集满足最小支持度的项，找出频繁1项集的集合，该集合称作L_1。
- 然后，L_1用于找频繁2项集的集合L_2，L_2用于寻找L_3，如此下去，直到不能再找到新的频繁k项集。找每个L_k要求对数据库做一次完全扫描。
- 利用频繁项集构造出满足用户最小信任度的规则。

算法核心--生成所有频繁项集



生成所有频繁项目集-思路

◆**基本思想：逐级搜索**（Level-wise Search）

先找出长度为1的频繁项目集，接着找出长度为2的频繁项目集，以此类推

$$L_1 \Rightarrow C_2 \Rightarrow L_2 \Rightarrow C_3 \Rightarrow L_3 \Rightarrow C_4 \Rightarrow L_4 \Rightarrow \cdots\cdots$$

- **由频繁(k−1)−项集构建候选k−项集：**
  - **找到所有的频繁1−项集**
  - **扩展频繁(k−1)−项集得到候选k−项集**
  - **剪除不满足最小支持度的候选项集**



Apriori 算法伪代码

C_k: Candidate itemset of size k

L_k: Frequent itemset of size k

$L_1 \leftarrow \{frequent\ items\}$

for (k=1; $L_k \neq \emptyset$; k++)

  $C_{k+1} \leftarrow candidate(L_k)$ — candidates

  for each transaction t

    $Q \leftarrow \{c \mid c \in C_{k+1} \wedge c \subseteq t\}$

    $count[c] \leftarrow count[c] + 1, \quad \forall c \in Q$ — counting

  end for

  $L_{k+1} \leftarrow \{c \mid c \in C_{k+1} \wedge count[c] / N \geq \sigma\}$ — filtering

end for

return $\bigcup_k L_k$



Apriori 算法总结

- 生成特定大小的项目集
- 扫描数据库一次，看看哪一个是频繁的.
- 使用频繁项集生成为 size = size+1 的候选项集.
- 迭代地查找基数从 1 到 k 的频繁项集.
- 避免产生不常见罕见的候选项集.
- 需要对数据库进行多次扫描.
- 高效的索引技术对算法的改进会有很大的帮助.



实例（$L_k \rightarrow C_{k+1}$）

$$L_1 = \{1, 2, 3, 4, 5\} \quad L_2 = \{\{1, 2\}, \{2, 3\}\}$$

$$\{X \cup p \mid X \in L_k, p \in L_1, p \notin X\}$$

$$C_3 = \{\{1, 2, 3\}, \{1, 2, 4\}, \{1, 2, 5\}, \{2, 3, 4\}, \{2, 3, 5\}\}$$

$$\{X \cup Y \mid X, Y \in L_k, \mid X \cap Y \mid = k - 1\}$$

$$C_3 = \{\{1, 2, 3\}\}$$





实例（$L_k \rightarrow C_{k+1}$）

$$\{X \bigcup Y_k \mid X, Y \in L_k, X_i = Y_i, \forall i \in [1, k-1], X_k \neq Y_k\}$$

Ordered List

$L_2 = \{\{1,2\},\{2,3\}\}$ $C_3 = \{\}$

$L_2 = \{\{1,3\},\{2,3\}\}$ $C_3 = \{\}$



$L_2 = \{\{1,2\},\{1,3\},\{2,3\}\}$ $C_3 = \{\{1,2,3\}\}$

$L_2 = \{\{1,2\},\{1,3\}\}$ ✗ $C_3 = \{\{1,2,3\}\}$





验证正确性

$$\forall X, X \in L_{k+1} \Rightarrow X \in C_{k+1}$$

$$\{X_1, \ldots, X_k, X_{k+1}\} \in L_{k+1}$$

$$\Downarrow$$

$$\{X_1, \ldots, X_{k-1}, X_k\} \in L_k$$

$$\{X_1, \ldots, X_{k-1}, X_{k+1}\} \in L_k$$

Join

$$\{X_1, \ldots, X_{k-1}, X_k, X_{k+1}\} \in C_{k+1}$$





实例（$L_k \rightarrow C_{k+1}$）

- L_3 = {{1, 2, 3}, {1, 2, 4}, {1, 3, 4}, {1, 3, 5}, {2, 3, 4}}

- After join：
  - C_4 = {{1, 2, 3, 4}, {1, 3, 4, 5}}
- After pruning：
  - L_4 = {{1, 2, 3, 4}}

    because {1, 4, 5} is not in L_3 ({1, 3, 4, 5} is removed)



Apriori 算法—示例

Support ≥ 2

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

1st scan →

C_1

Itemset	sup
{A}	2
{B}	3
{C}	3
~~{D}~~	~~1~~
{E}	3

→ L_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

C_2

Itemset
{A, B}
{A, C}
{A, E}
{B, C}
{B, E}
{C, E}

2nd scan →

C_2

Itemset	sup
~~{A, B}~~	~~1~~
{A, C}	2
~~{A, E}~~	~~1~~
{B, C}	2
{B, E}	3
{C, E}	2

→ L_2

Itemset	sup
{A, C}	2
{B, C}	2
{B, E}	3
{C, E}	2

C_3

Itemset
{B, C, E}

3rd scan →

L_3

Itemset	sup
{B, C, E}	2



Apriori 算法的重要细节

- 如何产生候选?
  - 步骤 1: L_k的自连接
  - 步骤 2: 剪枝
- 如何对候选的支持度计数?
- 候选产生的例子
  - L_3={*abc, abd, acd, ace, bcd*}
  - 自连接: L_3*L_3
    - *abcd*：由 *abc* 和 *abd*
    - *acde*：由 *acd* 和 *ace*
  - 剪枝:
    - *acde* 被删除, 因为 *ade* 不在 L_3
  - C_4={*abcd*}



Clothing Example

Apriori-Gen Algorithm – Clothing Example

- Given: 20 clothing transactions; *s*=20%, c=50%
- Generate association rules using the Apriori algorithm

Transaction	Items	Transaction	Items
t_1	Blouse	t_{11}	TShirt
t_2	Shoes, Skirt, TShirt	t_{12}	Blouse, Jeans, Shoes, Skirt, TShirt
t_3	Jeans, TShirt	t_{13}	Jeans, Shoes, Shorts, TShirt
t_4	Jeans, Shoes, TShirt	t_{14}	Shoes, Skirt, TShirt
t_5	Jeans, Shorts	t_{15}	Jeans, TShirt
t_6	Shoes, TShirt	t_{16}	Skirt, TShirt
t_7	Jeans, Skirt	t_{17}	Blouse, Jeans, Skirt
t_8	Jeans, Shoes, Shorts, TShirt	t_{18}	Jeans, Shoes, Shorts, TShirt
t_9	Jeans	t_{19}	Jeans
t_{10}	Jeans, Shoes, TShirt	t_{20}	Jeans, Shoes, Shorts, TShirt

- Scan1: Find all 1-itemsets. Identify the frequent ones.

  Candidates: Blouse, Jeans, Shoes, Shorts, Skirt, Tshirt

  Support: 3/20 14/20 10/20 5/20 6/20 14/20

  Frequent (Large): Jeans, Shoes, Shorts, Skirt, Tshirt

  Join the frequent items – combine items with each other to generate candidate pairs



Clothing Example

Clothing Example – cont.1

Jeans, Shoes, Shorts, Skirt, Tshirt

- **Scan2: 10 candidate 2-itemsets were generated. Find the frequent ones.**

{Jeans, Shoes}:7/20 {Shoes, Short}:4/20 ~~{Short, Skirt}: 0/20~~ {Skirt, TShirt}: 4/20

{Jeans, Short} :5/20 ~~{Shoes, Skirt}: 3/20~~ {Short, TShirt}: 4/20

~~{Jeans, Skirt}~~ :3/20 {Shoes, TShirt}: 10/20

{Jeans, TShirt}:9/20 4/20 *7 frequent itemsets are found out of 10.*

Scan	Candidates	Large Itemsets
1	{Blouse}, {Jeans}, {Shoes}, {Shorts}, {Skirt}, {TShirt}	{Jeans}, {Shoes}, {Shorts} {Skirt}, {Tshirt}
2	{Jeans, Shoes}, {Jeans, Shorts}, {Jeans, Skirt}, {Jeans, TShirt}, {Shoes, Shorts}, {Shoes, Skirt}, {Shoes, TShirt}, {Shorts, Skirt}, {Shorts, TShirt}, {Skirt, TShirt}	{Jeans, Shoes}, {Jeans, Shorts}, {Jeans, TShirt}, {Shoes, Shorts}, {Shoes, TShirt}, {Shorts, TShirt}, {Skirt, TShirt}
3	{Jeans, Shoes, Shorts}, {Jeans, Shoes, TShirt}, {Jeans, Shorts, TShirt}, {Jeans, Skirt, TShirt}, {Shoes, Shorts, TShirt}, {Shoes, Skirt, TShirt}, {Shorts, Skirt, TShirt}	{Jeans, Shoes, Shorts}, {Jeans, Shoes, TShirt}, {Jeans, Shorts, TShirt}, {Shoes, Shorts, TShirt}
4	{Jeans, Shoes, Shorts, TShirt}	{Jeans, Shoes, Shorts, TShirt}
5	Ø	Ø

Everyone is combined with each other

2 sets are joined if they have 1 item in common (i,.e. 1 item different)

2 sets are joined if they have 2 item in common (i,.e. 1 item different)



Clothing Example

Clothing Example – cont.2

- The next step is to use the large itemsets and generate association rules
- c=50%
- The set of large itemsets is

L={{Jeans},{Shoes}, {Shorts}, {Skirt}, {TShirt}, {Jeans, Shoes}, {Jeans, Shorts}, {Jeans, TShirt}, {Shoes, Shorts}, {Shoes, TShirt}, {Shorts, TShirt}, {Skirt, TShirt}, {Jeans, Shoes, Shorts}, {Jeans, Shoes, TShirt}, {Jeans, Shorts, TShirt},{Shoes, Shorts, TShirt}, {Jeans, Shoes, Shorts,TShirt} }

- We ignore the first 5 as they do not consists of 2 nonempty subsets of large itemsets. We test all the others, e.g.:

$$Confidence(Jeans \rightarrow Shoes) = \frac{Support(\{Jeans, Shoes\})}{Support(\{Jeans\})} = \frac{7/20}{14/20} = 50\% \geq c$$



Apriori的性能瓶颈问题

1）多次扫描事务数据库，需要很大的I/O负载

$$C_k \Rightarrow L_k$$

2）可能产生庞大的候选集

$$L_{k-1} \Rightarrow C_k$$

比如：10^4个L_1有近10^7个C_2



关联规则挖掘误区（Myth No. 1）

- 一个强规则并不代表它有意义
- **例:**
  - |D|=10000
  - #{DVD}=7500
  - #{Tape}=6000
  - #{DVD, Tape}=4000
  - Thresholds: σ = 30%, Φ = 50%
  - Support(Tape → DVD)= 4000/10000=40%
  - Confidence(Tape → DVD)=4000/6000=66%



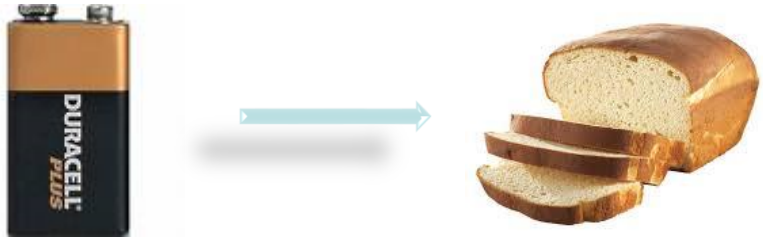
- **综上我们得到强关联规则:** Tape → DVD
  - 看来录像带的销售很利于推广DVD
  - 但是, **P(DVD)=75% > P(DVD | Tape) !!**
  - 所以得出购买录像带的购买者不太可能去买DVD



关联规则挖掘误区（Myth No. 2）

Transactions
Bread, Milk
Bread, Battery
Bread, Butter
Bread, Honey
Bread, Chips
Yogurt, Coke
Bread, Battery
Cookie, Jelly

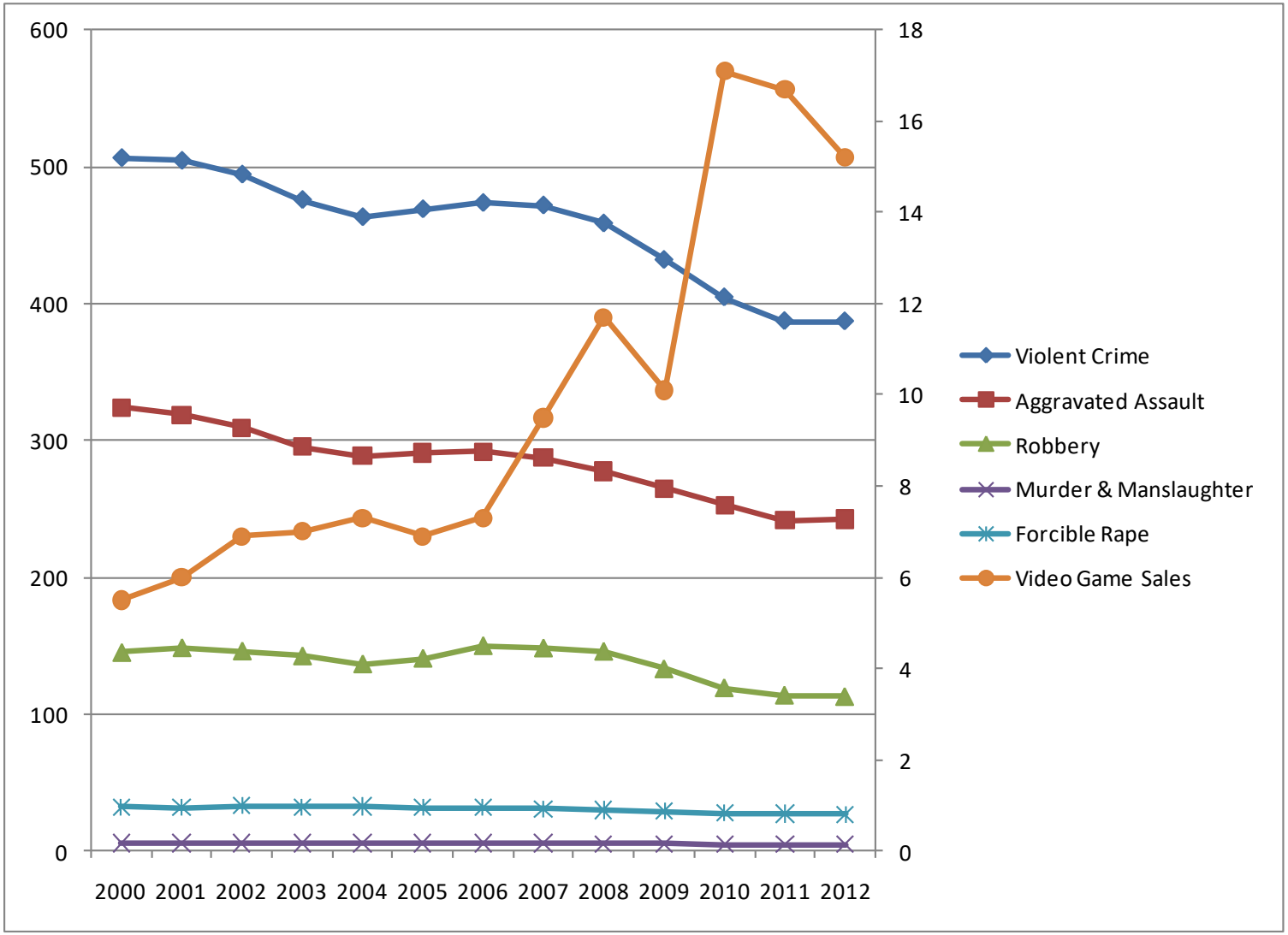
$$P(Bread \mid Battery) = 100\% > P(Bread) = 75\%$$







关联规则挖掘误区（Myth No. 3）







关联规则挖掘误区（Myth No. 3）

Association ≠ Causality







切记：我们所有运用数据挖掘分析的关联规则仅仅代表的是条件概率问题*P(Y|X)*



概念总结:

- 支持度s: 事务包含 $X \cup Y$ 的概率
- 置信度c: 事务含 X 也包含 Y 的条件概率.
- 强规则: 同时满足 最小支持度阈值和最小置信度阈值的规则
- **频繁项集（Frequent Itemsets）**：事务数据中支持度大于最小支持度阈值的所有项集
- **关联规则（Association Rule）**：事务数据中支持度大于最小支持度阈值且置信度大于最小置信度阈值的所有规则



关联规则作业

1. 关联规则定义。
2. 支持度和置信度定义和其计算方法。
3. 什么是频繁项集，它具有哪些性质?
4. Apriori算法的思想和核心各是什么?
5. 简述Apriori算法的不足之处。

本次作业下周一(11/8)提交